Projektoversigt – Secure Coding Webshop

Samlet teknisk overblik over arkitektur, kode, datahåndtering og sikkerhed

1. Formål og afgrænsning

Secure Coding Webshop er en mindre skolewebshop udviklet i **C# og .NET 9**. Projektet demonstrerer en enkel, lagdelt webapplikation med fokus på grundlæggende sikkerhed, tydelig ansvarsfordeling og kode, der er til at gennemskue.

Brugeren kan blandt andet:

- oprette en konto og logge ind,
- søge efter produkter,
- lægge produkter i en indkøbskurv,
- gennemføre et simuleret køb,
- se egne tidligere ordrer,
- ændre sin adgangskode.

En administrator kan desuden oprette, redigere og deaktivere produkter.

Projektet er bevidst holdt simpelt. Det indeholder ingen rigtig betalingsløsning, eksterne API'er, e-mailudsendelse, ORM eller avanceret JavaScript. Brugergrænsefladen fungerer med almindelige Razor Pages-formularer og server-side behandling.

2. Teknologier

Område	Teknologi	Anvendelse
Platform	.NET 9	Kørsel af hele applikationen
Web	ASP.NET Core Razor Pages	Sider, formularer og HTTP-handlers
Sprog	C#	Forretningslogik og dataadgang
Database	Microsoft SQL Server LocalDB	Lokal lagring af brugere, produkter og ordrer
Dataadgang	ADO.NET / `Microsoft.Data.SqlClient`	Direkte, parameteriserede SQL-forespørgsler
Login	ASP.NET Core Cookie Authentication	Opretholder brugerens login mellem requests
Passwords	`PasswordHasher<TUser>`	Sikker hashing og kontrol af adgangskoder
Kurv	ASP.NET Core Session	Midlertidig lagring af produkt-id og antal
Brugerflade	Razor, HTML og CSS	Server-renderede sider uden JavaScript

3. Løsningsstruktur og arkitektur

Løsningen består af to aktive projekter:

```
SecureCodingWebshop.slnx
│
├── src/Webshop.UI
│   ├── Pages
│   ├── Security
│   ├── Services
│   ├── wwwroot
│   └── Program.cs
│
├── src/Webshop.Library
│   ├── Data
│   ├── Models
│   └── Services
│
└── database.sql
```

Webshop.UI

`Webshop.UI` er præsentationslaget. Det modtager brugerens HTTP-requests, viser Razor Pages og kalder den nødvendige logik i library-projektet.

Projektet har ansvar for:

- visning og behandling af formularer,
- navigation og beskeder til brugeren,
- cookie-login og adgangskontrol,
- sessionsbaseret indkøbskurv,
- registrering af services og middleware,
- sikkerheds-headere på HTTP-svar.

Webshop.Library

`Webshop.Library` indeholder de dele af applikationen, som ikke er direkte knyttet til brugergrænsefladen.

Projektet har ansvar for:

- domænemodeller som bruger, produkt og ordre,
- regler for adgangskoder,
- registrering og login af brugere,
- databaseforbindelser,
- alle SQL-forespørgsler og transaktioner.

Afhængigheder mellem lagene

Afhængigheden går fra brugergrænsefladen til biblioteket:

```
Browser
   ↓
Razor Page / PageModel
   ↓
Service eller StoreRepository
   ↓
SqlConnectionFactory
```

```
    ↓
SQL Server LocalDB
```

`Webshop.Library` kender ikke til de konkrete Razor Pages. Det gør det lettere at forstå, teste og genbruge den centrale logik.

4. Opstart og konfiguration

Program.cs

`Program.cs` samler og konfigurerer applikationen. Filen:

- registrerer Razor Pages,
- angiver hvilke mapper der kræver login eller administratorrolle,
- konfigurerer login-cookien,
- konfigurerer sessionen til indkøbskurven,
- registrerer repository, services og password-hasher,
- aktiverer middleware i den korrekte rækkefølge,
- tilføjer sikkerheds-headere,
- forbinder Razor Pages til applikationen.

Følgende områder er beskyttet:

Område	Krav
`/Orders`	Brugeren skal være logget ind
`/Checkout`	Brugeren skal være logget ind
`/Account/ChangePassword`	Brugeren skal være logget ind
`/Admin`	Brugeren skal have rollen `Admin`

Databaseforbindelsen læses først fra konfigurationen. Hvis den ikke findes, bruges den lokale udviklingsdatabase:

```
Server=(localdb)\MSSQLLocalDB;Database=SecureCodingWebshop
```

5. Modeller

Modellerne beskriver de data, der bevæger sig mellem database, services og brugergrænseflade.

Model	Ansvar
`AppUser`	Indeholder bruger-id, e-mail, password-hash, rolle og oplysninger om loginspærring
`Product`	Beskriver navn, beskrivelse, pris, lagerstatus og om produktet er aktivt
`CartLine`	Gemmer kun produkt-id og antal i sessionen
`CartDisplayLine`	Samler aktuelle produktdata til visning af kurven
`OrderSummary`	Viser en kort oversigt over en ordre
`OrderLineView`	Beskriver én varelinje i en ordre
`OrderDetails`	Samler ordreoplysninger og alle tilhørende varelinjer

Model	Ansvar
`AuthenticationResult`	Returnerer resultat, bruger og besked efter et loginforsøg

Der skelnes mellem data, som gemmes i kurven, og data, som vises. Kurven gemmer ikke produktnavn eller pris. Disse oplysninger hentes på ny fra databasen, så brugeren ikke kan ændre prisen gennem sessionsdata.

6. Services og central logik

SqlConnectionFactory

`SqlConnectionFactory` opretter nye SQL-forbindelser ud fra den konfigurerede connection string. Klassen åbner ikke selv forbindelsen; det gør den metode, som skal bruge databasen.

Det samler oprettelsen af forbindelser ét sted og betyder, at repository-klassen ikke behøver kende detaljerne om konfigurationen.

StoreRepository

`StoreRepository` er projektets dataadgangslag. Det er den eneste klasse, der udfører SQL mod databasen.

De vigtigste metodegrupper er:

Område	Metoder	Ansvar
Produkter	`GetProductsAsync`, `GetProductAsync`	Henter produktliste eller et bestemt produkt
Produktadministration	`SaveProductAsync`, `DeactivateProductAsync`	Opretter, redigerer eller deaktiverer produkter
Brugere	`FindUserByEmailAsync`, `CreateUserAsync`	Finder og opretter brugere
Login	`RegisterFailedLoginAsync`, `ResetFailedLoginAsync`	Tæller fejlede forsøg og håndterer spærring
Password	`UpdatePasswordAsync`	Gemmer et nyt password-hash
Ordrer	`CreateOrderAsync`, `GetOrdersAsync`, `GetOrderAsync`	Opretter og henter brugerens ordrer
Audit	`WriteAuditAsync`	Gemmer relevante sikkerheds- og brugerhændelser

Alle værdier sendes til SQL som typede parametre. Brugerinput bliver derfor ikke sat direkte ind i SQL-teksten.

UserService

`UserService` samler den forretningslogik, der vedrører brugere.

RegisterAsync

Metoden:

1. normaliserer e-mailadressen,
2. kontrollerer e-mail og passwordpolitik,
3. undersøger om e-mailen allerede findes,
4. hasher adgangskoden,
5. opretter brugeren,

6. skriver hændelsen til auditloggen.

Adgangskoden gemmes aldrig i klartekst.

AuthenticateAsync

Metoden:

1. finder brugeren ud fra e-mail,
2. kontrollerer om kontoen er midlertidigt spærret,
3. verificerer password-hashet,
4. registrerer et fejlet forsøg eller nulstiller tælleren,
5. skriver loginforsøget til auditloggen,
6. returnerer brugeroplysningerne ved korrekt login.

Ukendt e-mail og forkert adgangskode giver den samme almindelige fejlbesked. Det gør det vanskeligere at undersøge, hvilke e-mailadresser der findes i systemet.

ChangePasswordAsync

Metoden kontrollerer først den nuværende adgangskode. Derefter valideres den nye adgangskode, hashes og gemmes. Ændringen registreres i auditloggen.

PasswordPolicy

`PasswordPolicy` indeholder projektets adgangskoderegler. En adgangskode skal:

- være mindst 12 tegn,
- indeholde mindst ét stort bogstav,
- indeholde mindst ét lille bogstav,
- indeholde mindst ét tal,
- indeholde mindst ét specialtegn.

`Validate` returnerer de krav, der ikke er opfyldt. `Score` beregner en enkel styrkescore fra 0 til 100. Begge dele kører på serveren.

CartSession

`CartSession` håndterer indkøbskurven i ASP.NET Core Session.

Klassen kan:

- hente kurven,
- tilføje et produkt,
- ændre antal,
- fjerne et produkt,
- tømme kurven,
- hente aktuelle produktdata til visning.

Antallet begrænses til mellem 1 og 20. Sessionen indeholder kun produkt-id og antal. Pris, navn og lager hentes fra databasen, når kurven skal vises eller gennemføres.

ClaimsExtensions

`ClaimsExtensions.UserId` henter den indloggede brugers id fra brugerens claims. Dermed behøver PageModels ikke selv gentage koden til at læse og konvertere id'et.

7. Razor Pages og brugerfunktioner

En Razor Page består typisk af:

- en `.cshtml`-fil med HTML og Razor-markup,
- en `.cshtml.cs`-fil med en `PageModel` og sidens server-side logik.

Handler-metoder følger navne som `OnGet`, `OnPost` og `OnPostAdd`. ASP.NET Core vælger automatisk den korrekte metode ud fra HTTP-metoden og formularens handler.

Produktkatalog – `/Index`

Forsiden henter aktive produkter og understøtter en søgetekst på højst 60 tegn. Søgningen udføres med en parameteriseret `LIKE`-forespørgsel. Et produkt kontrolleres i databasen, før det lægges i kurven.

Registrering – `/Account/Register`

Siden modtager e-mail, adgangskode og bekræftelse. Data annotations udfører den første formularvalidering, mens `UserService` håndterer de egentlige bruger- og passwordregler.

Login og logout – `/Account/Login`

Ved korrekt login oprettes claims med brugerens id, e-mail og rolle. Claims gemmes i en sikker authentication-cookie. Logout registrerer hændelsen og fjerner cookien.

Ændring af adgangskode – `/Account/ChangePassword`

Siden beregner passwordscore på serveren og kalder `UserService.ChangePasswordAsync`. Brugeren skal kende sin nuværende adgangskode.

Kurv – `/Cart`

Kurvesiden viser aktuelle produktdata og den samlede pris. Formularer bruges til at ændre antal, fjerne varer og tømme kurven. Efter en ændring omdirigeres brugeren tilbage til siden, så browseren ikke gensender formularen ved opdatering.

Checkout – `/Checkout`

Checkout viser kurven og gennemfører en lokal simulation. Der sendes ingen betaling. Ved et gyldigt køb oprettes ordren, kurven tømmes, og brugeren sendes til ordredetaljerne.

Hvis lageret eller andre data ikke længere er gyldige, rulles købet tilbage, og brugeren får en fejlbesked.

Ordrer – `/Orders`

Ordreoversigten henter kun ordrer for bruger-id'et i login-cookien. Detaljesiden filtrerer både på ordre-id og bruger-id. En bruger kan derfor ikke få adgang til en anden brugers ordre ved blot at ændre id'et i URL'en.

Produktadministration – `/Admin/Products`

Hele adminmappen kræver rollen `Admin`. Administratoren kan:

- se aktive og deaktiverede produkter,
- oprette produkter,

- redigere produktdata,
- deaktivere produkter.

Produkter deaktiveres frem for at blive slettet. Tidligere ordrelinjer og referencer til produktet kan dermed bevares.

8. Database og datahåndtering

Databasen hedder `SecureCodingWebshop` og oprettes af `database.sql`.

Tabeller

Tabel	Formål	Vigtige relationer og regler
`Product`	Gemmer webshoppens produkter	Pris skal være positiv, og lager må ikke være negativt
`AppUser`	Gemmer brugerkonti	E-mail er unik, og rolle skal være `Customer` eller `Admin`
`ShopOrder`	Gemmer ordrens hovedoplysninger	Hver ordre tilhører én bruger
`OrderLine`	Gemmer varerne på en ordre	Hver linje tilhører en ordre og et produkt
`AuditEvent`	Gemmer udvalgte hændelser	Kan knyttes til en bruger, men understøtter også anonyme hændelser

Centrale relationer

```
AppUser   1 ── * ShopOrder
ShopOrder 1 ── * OrderLine
Product   1 ── * OrderLine
AppUser   1 ── * AuditEvent
```

`OrderLine` gemmer produktnavn og enhedspris på købstidspunktet. En gammel ordre ændrer sig derfor ikke, hvis produktets navn eller pris senere redigeres.

Constraints

Databasen indeholder regler, som beskytter data, selv hvis en fejl skulle slippe gennem applikationen:

- unikke e-mailadresser,
- kun kendte brugerroller,
- positive produkt- og ordrelinjepriser,
- lager på mindst nul,
- antal mellem 1 og 20,
- fremmednøgler mellem brugere, ordrer, produkter og ordrelinjer.

Parameteriseret SQL

Repository-metoderne bruger `SqlParameter` med konkrete SQL-typer og længder. Det har to formål:

- at beskytte mod SQL injection,
- at sikre forudsigelig konvertering mellem C#- og SQL-datatyper.

Checkout-transaktionen

`CreateOrderAsync` udfører checkout som én samlet SQL-transaktion:

1. Kurven kontrolleres.

2. Hvert produkt hentes og låses med `UPDLOCK`.
3. Lager og antal valideres.
4. Den samlede pris beregnes fra databasepriserne.
5. Ordren oprettes.
6. Ordrelinjerne oprettes.
7. Lageret reduceres.
8. Transaktionen godkendes med `Commit`.

Hvis et trin fejler, udføres `Rollback`. Dermed undgås halve ordrer og lagerændringer uden en tilhørende ordre.

9. Login, session og autorisation

Authentication

Authentication afgør, hvem brugeren er. Efter login indeholder cookien claims med:

- bruger-id,
- navn/e-mail,
- rolle.

Cookien er konfigureret med:

- `HttpOnly`, så browserens scripts ikke kan læse den,
- `Secure`, så den kun sendes over HTTPS,
- `SameSite=Lax`, som begrænser cross-site afsendelse,
- 30 minutters udløbstid,
- sliding expiration ved fortsat aktivitet.

Authorization

Authorization afgør, hvad den indloggede bruger må gøre. Almindelige kunder kan se egne ordrer og gennemføre checkout. Kun rollen `Admin` kan åbne administrationssiderne.

Session

Sessionen bruges til kurven og har sin egen cookie. Cookien identificerer sessionen, mens selve kurvdataene opbevares på serveren. Kurven udløber efter to timers inaktivitet.

10. Sikkerhedskontroller

Projektets vigtigste sikkerhedsvalg er:

Risiko	Kontrol i projektet
Passwords lækkes	Passwords hashes og gemmes aldrig i klartekst
Password-gætteri	Passwordpolitik og 15 minutters spærring efter fem fejl
SQL injection	Alle brugerdata sendes som SQL-parametre
Uautoriseret adminadgang	Rollepolicy på hele `/Admin`-mappen

Risiko	Kontrol i projektet
Adgang til andres ordrer	SQL filtrerer på både ordre-id og bruger-id
Manipulation af pris	Pris og lager genlæses fra databasen ved checkout
Halv ordre	Checkout udføres i én transaktion
CSRF	Razor Pages anvender antiforgery-beskyttelse på formularer
XSS	Razor encoder output som standard, og CSP begrænser indhold
Clickjacking	`X-Frame-Options: DENY`
Følsomme data i audit	Audit gemmer hændelser, men ikke passwords eller cookies

11. Gennemgående programflows

Loginflow

```
Loginformular
   ↓
LoginModel.OnPostAsync
   ↓
UserService.AuthenticateAsync
   ↓
StoreRepository + PasswordHasher
   ↓
Claims og authentication-cookie
```

Kurvflow

```
Produkt vælges på forsiden
   ↓
IndexModel.OnPostAddAsync
   ↓
CartSession.Add
   ↓
Produkt-id og antal gemmes i session
   ↓
Aktuelle data hentes fra databasen ved visning
```

Checkoutflow

```
Brugeren godkender checkout
   ↓
Checkout PageModel
   ↓
StoreRepository.CreateOrderAsync
   ↓
SQL-transaktion kontrollerer lager og opretter ordre
   ↓
Kurven tømmes og ordren vises
```

12. Kørsel af projektet

1. Kør `database.sql` mod `(localdb)\MSSQLLocalDB`.
2. Gendan projektets pakker:

   ```
   dotnet restore SecureCodingWebshop.slnx
   ```

3. Start webprojektet:

```
dotnet run --project src/Webshop.UI
```

4. Åbn den HTTPS-adresse, som vises i terminalen.

En almindelig bruger oprettes gennem registreringssiden. Brugeren kan efterfølgende gives administratorrollen med den kommenterede SQL-kommando nederst i `database.sql`.

13. Projektets vigtigste designvalg

- **Razor Pages frem for en tung frontend:** Færre bevægelige dele og ingen nødvendig JavaScript.
- **Ét repository:** Dataadgangen er samlet og nem at finde i et mindre skoleprojekt.
- **ADO.NET frem for ORM:** SQL og sikker parameterisering er synligt i koden.
- **Session til kurven:** Kurven behøver ikke en separat databasetabel før checkout.
- **Serveren er autoritativ:** Pris, lager, bruger-id og rettigheder afgøres på serveren.
- **Deaktivering frem for sletning:** Historiske data og relationer bevares.
- **Simpel checkout-simulation:** Projektet demonstrerer ordrelogik uden betalingsudbyder og eksterne hemmeligheder.

14. Supplerende dokumenter

Projektet indeholder også:

- `README.md` – kort opsætning og afgrænsning,
- `SECURITY.md` – konkrete sikkerhedsvalg og manuelle kontroller,
- `THREAT_MODEL.md` – risici, kontroller og adgangsmatrix,
- `TRACEABILITY.md` – kobling mellem user stories og implementering,
- `database.sql` – komplet oprettelse af database og testprodukter.